

RECOMMENDED AI MODEL: GPT-6 Astra Advanced

TASK: FULL-UNITIZED MINI-STAAD ENHANCEMENT, NUMERICAL DEBUGGING, AND
OPENSTAAD REFERENCE MATCHING

You are working on our in-house Mini-STAAD structural-analysis engine used by the NextGen Facade application.

Your objective is to enhance and correct the Mini-STAAD implementation specifically for the FULLY UNITIZED facade flow so that it:

- 1. Correctly handles every valid Full-Unitized structural model generated by our application, including user-customized geometry, panel/bay counts, dimensions and PRIS properties.
- 2. Correctly analyzes the actual generated STAAD .std model.
- 3. Produces structural results that closely reproduce the behavior/results obtained from STAAD.Pro through OpenSTAAD for the supported model class.
- 4. Fixes the currently observed discrepancies, especially bending moment and deflection/displacement.
- 5. Does not rely on sample-specific hardcoding, arbitrary correction factors, or JSON-level hacks.

This is a structural engineering and numerical-analysis task, not merely a JSON comparison task.

=====

1. CURRENT REFERENCE FLOW

=====

Our current reference flow is:

NextGen Facade



Full-Unitized user inputs



STAAD .std generation



STAAD.Pro



OpenSTAAD API



Reference extraction JSON

The in-house flow is:

NextGen Facade



Full-Unitized user inputs



STAAD .std generation



Mini-STAAD / in-house solver



Extraction JSON

The supplied OpenSTAAD reference JSON contains results including:

- global bending moment
- global shear force
- global axial force
- global displacement
- profile-specific bending moment
- profile-specific shear
- profile-specific axial
- profile-specific displacement
- mullion

- stack
- head
- sill
- transom
- governing member IDs
- governing load cases
- governing stations
- local force axes
- member type
- load case name
- units
- envelope information

Do not assume that the JSON field name alone defines the engineering semantics. Verify semantics from the repository, reference behavior, and supplied models.

=====

2. KNOWN PROBLEM

=====

For the provided Full-Unitized model, Mini-STAAD and OpenSTAAD produce slightly different results.

The observed differences are primarily in:

- Bending Moment
- Deflection / Displacement

These discrepancies can occur in:

- properties.global
- properties.mullion

- properties.stack
- properties.head
- properties.sill
- properties.transom

Example reference values include:

Global major bending:

value = 3.539295 kN-m
member_id = 17
load_case = 2
station = 32.086548 in
axis = MZ

Global resultant displacement:

value = 31.987816 mm
load_case = 3
source = member-span
member_id = 17
station = 32.086548 in

These are REFERENCE VALUES ONLY.

NEVER hardcode them.

=====

3. VERY IMPORTANT: FULL-UNITIZED MODELS ARE USER-CUSTOMIZABLE

=====

The Full-Unitized UI is NOT a fixed structural model.

The generated .std varies according to the user's input configuration.

The screenshots supplied with this task show that the user can customize, among other things:

- number of bays
- bay dimensions
- number of panels
- panel/segment dimensions
- stack dimensions
- PRIS/member properties
- profile assignments

Therefore Mini-STAAD MUST NOT be built around one fixed Full-Unitized geometry.

The real target is the full class of valid Full-Unitized .std models that our application can generate.

=====

4. VARIABLE NUMBER OF BAYS

=====

The UI allows the user to change the number of bays.

For example:

Bays = 3

is only one configuration.

Other supported configurations may include:

Bays = 1

Bays = 2

Bays = 3

Bays = 4

Bays = 5

...

Determine the actual supported limits from the repository.

Bay dimensions are also customizable.

Example:

B1 = 1.30 m

B2 = 1.30 m

B3 = 1.30 m

may become:

B1 = 1.20 m

B2 = 1.50 m

B3 = 1.35 m

B4 = 1.80 m

The solver must support:

- variable bay count
- unequal bay lengths
- arbitrary supported bay dimensions
- changing node topology
- changing member topology

- changing member lengths
- changing load distribution
- changing governing members
- changing governing stations

Never assume a fixed bay count.

Never assume bays are equal unless the actual .std says so.

=====

5. VARIABLE NUMBER OF PANELS

=====

The UI allows the user to add/remove panels.

Example:

Panel 1

Panel 2

Panel 3

is only one configuration.

The actual generated model may contain other supported panel counts.

Changing panel count can alter:

- node count
- member count
- member connectivity
- transom count

- geometry
- panel load area
- load application positions
- member stations
- governing member
- governing displacement
- governing bending moment

Mini-STAAD must handle the actual .std topology dynamically.

Do not assume a fixed panel count.

=====

6. CUSTOM PANEL / SEGMENT DIMENSIONS

=====

The UI allows different dimensions such as:

S1
S2
S3
...

and other supported dimensions.

Example:

Panel 1:

S1 = 1.4 m

S2 = 3.4 m

Stack = 0.14 m

Panel 2:

S1 = 1.4 m

S2 = 3.8 m

S3 = 1.0 m

Stack = 0.14 m

The exact available fields must be identified from the application.

Mini-STAAD must calculate structural geometry from the actual generated .std.

Do NOT reconstruct geometry from frontend defaults.

Do NOT create a separate fixed geometry model inside Mini-STAAD.

=====

7. CUSTOM PRIS / MEMBER PROPERTIES

=====

The Full-Unitized UI includes Member Properties (PRIS).

The user can assign profile properties to member rows.

The UI demonstrates concepts such as:

MULLION

Male

Female

HEAD

SILL

MID TRANSON

STACK TRANSON

Some rows can be:

Auto-filled

Derived

Read-only

Some can be user assigned.

Mini-STAAD must process the actual structural properties contained in the generated .std.

Do NOT hardcode profile section properties.

=====

8. CUSTOM PROFILE ASSIGNMENTS

=====

The application uses a Profile Library.

A user can assign different profiles to different structural members.

Therefore two models generated from the same workflow can contain completely different section properties.

For example:

Model A:

A = A1

Ix = Ix1

Iy = Iy1

J = J1

Model B:

A = A2

Ix = Ix2

Iy = Iy2

J = J2

Mini-STAAD must calculate each model using its actual properties.

Never assume a profile name implies a fixed numeric property.

Never use frontend profile names as a substitute for structural properties if the actual structural values are available in .std.

=====

9. DERIVED PROFILES

=====

The application can expose derived/read-only profiles.

Examples include:

MULLION

Male + Female result

DERIVED

Read-only

and:

STACK TRANSON

DERIVED

Read-only

The solver should not depend on how those values were calculated in the UI.

Once the .std is generated, use the actual structural properties encoded in the model.

=====

10. .STD FILE IS THE STRUCTURAL SOURCE OF TRUTH

=====

Treat the actual generated .std as authoritative for the structural analysis.

The parser/model builder should derive:

- node IDs
- node coordinates
- member IDs
- start/end connectivity
- member length
- member orientation
- local coordinate system
- section properties
- material properties
- supports
- releases
- loads

- load cases
- load combinations
- all other supported structural information

Preferred pipeline:

Actual .std



STAAD parser



Canonical structural model



Validation



Unit normalization



Load/boundary assembly



Structural solver



Result recovery



Envelope/profile extraction



Existing result JSON

=====

11. NO FIXED TOPOLOGY

=====

Changes in bays/panels/properties can change:

- nodes
- members
- connectivity
- lengths
- supports
- load positions
- loads
- governing members
- governing stations

Mini-STAAD must therefore support dynamic structural topology.

Never rely on:

- fixed node count
- fixed member count
- fixed node IDs
- fixed member IDs
- fixed coordinates
- fixed element order
- fixed profile-to-member ranges

=====

12. NO HARDCODED MEMBER IDS

=====

The reference may report:

member_id = 17

Another model may report:

member_id = 42

or another value.

Never implement:

member 17 = mullion

member 45 = stack

member 55 = head

unless the mapping is derived from actual model semantics.

=====

13. NO HARDCODED NODE IDS

=====

Do not assume:

node 20 = head displacement

node 24 = stack displacement

node 27 = mullion displacement

Use the actual parsed structural model.

=====

14. MEMBER / PROFILE CLASSIFICATION

=====

The reference output includes profile/member types such as:

- mullion
- inner_mullion
- stack
- head
- sill
- transom
- horizontal

Determine how the existing application identifies these members.

Classification must remain valid when:

- node IDs change
- member IDs change
- bay count changes
- panel count changes
- member count changes
- profile assignments change

Preferred architecture:

Generic structural solver



Complete member result set



Facade/member classifier



Global envelope



Profile-specific envelopes

↓

JSON adapter

Do not embed fragile facade-specific ID ranges in the solver.

=====

15. LOADS MUST BE DYNAMIC

=====

Changing bay/panel geometry can alter load areas and structural load distribution.

Mini-STAAD must use the actual loads contained in each .std.

Inspect and correctly process:

- joint loads
- member loads
- uniform distributed loads
- partial distributed loads
- point loads
- moments
- local-direction loads
- global-direction loads
- load-case assignment

Do not assume the same load magnitude or load location across models.

=====

16. LOAD CASES

=====

The reference example includes load cases:

- 1
- 2
- 3
- 4

and names such as:

- DL
- WL

But future generated models must not be assumed to have exactly these cases.

Parse the actual .std.

Determine governing cases independently for:

- major bending
- minor bending
- major shear
- minor shear
- axial force
- displacement

=====

17. LOAD COMBINATIONS

=====

The reference extraction indicates:

```
load_case_mode = all
moment_mode = internal-envelope
include_combinations = true
```

Inspect how load combinations are represented in actual Full-Unitized .std files.

Verify:

1. Combination parsing.
2. Combination factors.
3. Correct case mapping.
4. Correct combined load vector.
5. Correct combined structural response.
6. No double counting.
7. Correct envelope semantics.
8. Correct governing load-case metadata.

Do not assume the largest individual case is always the governing result.

=====

18. STRUCTURAL ELEMENT FORMULATION

=====

Inspect the actual current Mini-STAAD formulation.

If the solver uses a 3D frame/beam element, verify the expected 12 DOF structure:

Node I:

UX UY UZ RX RY RZ

Node J:

UX UY UZ RX RY RZ

Verify:

- local stiffness matrix
- axial stiffness
- bending about both local axes
- torsion
- shear stiffness, if supported
- local coordinate system
- transformation matrix
- global matrix assembly
- boundary conditions
- matrix solution

Do not assume a simplified beam formula reproduces the actual reference behavior.

=====

19. TIMOSHENKO / EULER-BERNOULLI BEHAVIOR

=====

Inspect whether Mini-STAAD uses:

- Timoshenko
- Euler-Bernoulli
- hybrid formulation

Do not change the formulation just because one sample differs.

Determine which formulation is implemented now and whether it is appropriate for the actual .std models and expected reference behavior.

If shear areas are supported, verify:

- AY
- AZ
- shear correction factor
- G

If not supplied, verify the actual fallback behavior.

=====

20. SECTION PROPERTY INVESTIGATION

=====

Verify every PRIS/property required by the actual model:

- A
- Ix
- Iy
- Ixy where applicable
- J
- E
- G
- shear areas if applicable
- any other required property

Pay particular attention to:

MY

MZ

and mapping to local section axes/inertias.

A section-property or local-axis mistake can affect both moment and deflection.

=====

21. LOCAL / GLOBAL AXIS INVESTIGATION

=====

This is a major potential root cause.

Verify:

- local x
- local y
- local z
- beta angle
- orientation
- global-to-local load transformation
- local-to-global force recovery
- MY/MZ interpretation
- FY/FZ interpretation
- FX interpretation

Do not assume every member is aligned with a global axis.

Full-Unitized models can contain vertical and horizontal members with different orientations.

=====

22. SUPPORT CONDITIONS

=====

Read supports directly from each .std.

Verify:

- fixed
- pinned
- roller
- directional restraints
- partial restraints
- constrained DOFs

A support discrepancy can alter both displacement and bending.

Do not infer supports from expected JSON.

=====

23. MEMBER RELEASES

=====

Inspect Full-Unitized .std for member releases.

Correctly handle supported releases including:

- FX
- FY
- FZ
- MX
- MY
- MZ

A release mistake can change load path, stiffness, moment and deflection.

=====

24. BOUNDARY CONDITION IMPLEMENTATION

=====

Inspect how constrained DOFs are applied.

Verify that the mathematical treatment is correct and that reactions remain physically/equilibriumally consistent.

Do not change the method merely because outputs differ.

Find the actual cause.

=====

25. NUMERICAL DIAGNOSTICS

=====

Use available diagnostics such as:

- condition number
- minimum eigenvalue
- residual
- backward error
- equilibrium error
- singularity detection
- mechanism detection

A tiny residual only proves that the implemented equations were solved accurately. It does NOT prove the equations/model/loads correctly represent the STAAD reference.

Use numerical diagnostics to distinguish:

"solver accurately solved my model"

from:

"my model and formulation are equivalent to the intended reference behavior"

=====

26. BENDING MOMENT ROOT-CAUSE ANALYSIS

=====

For every mismatch compare:

Reference:

member

load case

station

axis

value

member type

Mini-STAAD:

member

load case

station

axis

value

member type

Identify the first stage where the results diverge.

Examples:

If loads match but displacement differs:

investigate stiffness/model/supports/releases

If displacement matches but internal moment differs:

investigate internal-force recovery

If internal forces match but reported governing result differs:

investigate station selection/envelope logic

Always fix the earliest incorrect layer.

=====

27. INTERNAL MEMBER FORCE RECOVERY

=====

The reference may identify a governing moment inside the member.

Example:

location = member-minimum

station = 32.086548 in

axis = MZ

Therefore endpoint-only checks may be insufficient.

Inspect how Mini-STAAD recovers:

- FX

- FY

- FZ
- MX
- MY
- MZ

along a member.

Determine whether it uses:

- end forces only
- fixed stations
- adaptive stations
- analytical functions
- exact extrema
- load-discontinuity boundaries
- left/right values around point loads

Implement the mathematically correct method for the supported element/load formulation.

=====

28. STATION / EXTREMA LOGIC

=====

Governing values can occur at:

- start
- end
- interior station
- local extremum
- immediately before a point load
- immediately after a point load

- other load discontinuity points

Inspect the current station sampling.

If continuous polynomial/analytical extrema can be determined, prefer that over coarse sampling where required for correctness.

Do not use arbitrary dense sampling merely to make a sample match without understanding the reference semantics.

=====

29. MEMBER-SPAN DEFLECTION

=====

The reference can identify:

source = member-span

rather than only:

source = node

Therefore internal member-span displacement must be implemented correctly.

Do not simply linearly interpolate end translations if the frame element's interpolation requires:

- end translations
- end rotations
- shape functions
- local deformation
- load-induced deformation
- local-to-global transformation

Investigate the exact current formulation and correct it as required.

=====

30. RESULTANT DISPLACEMENT

=====

Verify exactly how the reference defines:

RESULTANT

Do not silently assume its meaning.

A possible interpretation is:

$$\sqrt{UX^2 + UY^2 + UZ^2}$$

but verify this against actual reference models and current extraction logic.

=====

31. GLOBAL DISPLACEMENT

=====

The governing displacement may occur:

- at a node
- inside a member span

The solver must identify the governing location according to the verified semantics.

Do not assume the maximum is always nodal.

=====

32. PROFILE-LEVEL DISPLACEMENT

=====

For:

- mullion
- stack
- head
- sill
- transom

determine profile-specific governing displacement from the same solved structural model.

Do not run separate structural analyses for each profile type.

Preferred:

Solved global displacement field

↓

Member filtering

↓

Profile envelope

=====

33. GLOBAL VS PROFILE RESULTS

=====

The JSON contains:

properties.global
properties.mullion
properties.stack
properties.head
properties.sill
properties.transom

Determine the exact current semantics.

Likely:

global = all applicable members

profile category = filtered subset

But verify this from repository behavior.

The same underlying solver result should normally feed both.

=====

34. UNIT SYSTEM

=====

Audit the complete unit path:

.std units
↓
parser
↓
internal units
↓

stiffness/load calculations

↓

displacement

↓

internal forces

↓

output conversion

Reference output typically uses:

force = kN

length = mm

moment = kN-m

Station output may use a different displayed unit according to the established result contract.

Verify conversions for:

- force
- length
- moment
- E
- A
- I
- J
- loads
- displacement
- station

Do not scatter conversion constants through unrelated code.

Prefer centralized unit normalization.

=====

35. GLOBAL EQUILIBRIUM

=====

For each load case, validate global equilibrium after reactions:

$$\Sigma F \approx 0$$

$$\Sigma M \approx 0$$

Use this to validate:

- load assembly
- support reactions
- coordinate transformations
- equivalent nodal load vectors

Do not interpret equilibrium alone as proof of reference-equivalent behavior.

=====

36. FULL-UNITIZED GENERATOR COMPATIBILITY

=====

Inspect the existing application code that generates Full-Unitized .std files.

Trace:

Frontend inputs



geometry generation

↓
member generation
↓
PRIS assignment
↓
loads
↓
supports
↓
releases
↓
load cases
↓
combinations
↓
.std

Determine whether the generator already produces correct structural input.

If the generator is correct:

fix Mini-STAAD.

If generator has a defect:

document it separately.

Do not hide generator problems inside the solver.

=====
37. USER CUSTOMIZATION TEST MATRIX
=====

The supplied sample corpus may represent different configurations.

For each model identify actual characteristics such as:

- bays
- bay dimensions
- panels
- panel dimensions
- custom PRIS
- member count
- node count
- member categories
- section properties
- load cases
- combinations

Create a feature matrix using actual values.

Example:

Model	Nodes	Members	Bays	Panels	Custom PRIS	Notes
A	...	...	...	...	...	...
B	...	...	...	...	...	...
C	...	...	...	...	...	...

=====

38. MULTI-SAMPLE VALIDATION IS MANDATORY

=====

I will provide multiple:

*.std

and corresponding:

expected *.json

files for the Full-Unitized flow.

Use ALL of them.

Do not validate only the currently observed sample.

The models may vary in:

- bay count
- bay width/length
- panel count
- panel dimensions
- stack dimensions
- section properties
- profile assignment
- topology
- loads
- member IDs
- node IDs
- load cases
- combinations

The solver must handle those variations generically.

=====

39. BASELINE BEFORE CHANGES

=====

Before changing solver logic:

1. Run every supplied sample.
2. Save the current Mini-STAAD outputs.
3. Compare with reference.
4. Create a baseline report.
5. Identify discrepancies.

After changes:

1. Run every sample again.
2. Re-run the exact same comparison.
3. Measure improvement.
4. Confirm that no regression has been introduced.

=====

40. ERROR METRICS

=====

For reference R and actual A:

$\text{absolute_error} = \text{abs}(A - R)$

Use robust relative error:

$\text{relative_error} = \text{abs}(A - R) / \max(\text{abs}(R), \text{epsilon})$

Handle near-zero reference values properly.

Do not produce meaningless relative percentages for values near zero.

=====

41. TOLERANCES

=====

Do not choose arbitrary tolerances just to pass tests.

Determine appropriate tolerances based on:

- numerical formulation
- units
- conditioning
- reference behavior
- engineering precision
- required application accuracy

Use configurable thresholds.

Classify results, for example:

- PASS
- ACCEPTABLE
- INVESTIGATE
- FAIL

Document the basis for the tolerance.

=====

42. NO CORRECTION FACTORS

=====

NEVER do:

moment *= 0.97

deflection *= 1.03

or equivalent empirical adjustments.

Never implement:

if flow == FULLY_UNITIZED:

result *= correction

The correct solution must come from correcting the structural model, numerical formulation, load interpretation, result recovery or envelope logic.

=====

43. NO SAMPLE-SPECIFIC LOGIC

=====

NEVER do:

if filename == "USG_1.std":

...

NEVER do:

if memberId == 17:

...

NEVER do:

if nodeId == 20:

...

NEVER do:

if numberOfBays == 3:

special formula

unless the behavior represents an actual generic engineering/model rule supported by the input.

=====

44. NO HARDCODED EXPECTED RESULTS

=====

Never hardcode:

- moment
- shear
- axial
- displacement
- station
- member ID
- node ID
- profile property
- geometry
- load magnitude

- load case
- load combination

Everything must come from the actual model or actual generic solver logic.

=====

45. FIRST-DIVERGENCE DEBUGGING

=====

For every discrepancy classify the first incorrect layer:

PARSER
MODEL
UNIT
MATERIAL
SECTION PROPERTY
SUPPORT
RELEASE
LOAD
LOAD CASE
LOAD COMBINATION
STIFFNESS MATRIX
TRANSFORMATION
SOLVER
DISPLACEMENT RECOVERY
MEMBER FORCE RECOVERY
INTERNAL STATION
EXTREMA
ENVELOPE
PROFILE CLASSIFICATION
RESULT ADAPTER

Fix the earliest incorrect layer.

=====

46. DIAGNOSTIC INSTRUMENTATION

=====

If existing diagnostics are insufficient, add optional structured diagnostics for selected members/load cases.

Expose, as appropriate:

member ID

start node

end node

coordinates

length

local axes

orientation/beta angle

E

A

Ix

Iy

J

AY

AZ

loads

load case

end displacements

end rotations

local end forces

internal station

internal displacement

internal forces

Make diagnostics easy to enable/disable.

Do not change the production result contract unnecessarily.

=====

47. ENGINEERING BENCHMARKS

=====

Before relying exclusively on complex Full-Unitized comparisons, validate core solver behavior using simple deterministic cases where appropriate:

- axial member
- cantilever
- simply supported beam
- fixed-fixed beam
- distributed load
- point load
- internal moment extremum
- internal deflection extremum
- simple 2D frame
- simple 3D frame

Use known structural mechanics relationships.

This helps distinguish:

fundamental solver error

from:

Full-Unitized modeling/semantic error

=====

48. REGRESSION TESTS

=====

Every discovered defect must become a regression test.

Examples:

```
testFullUnitizedVariableBayCount()
testFullUnitizedUnequalBayDimensions()
testFullUnitizedVariablePanelCount()
testFullUnitizedCustomPanelDimensions()
testFullUnitizedCustomPrisProperties()
testFullUnitizedDynamicMemberTopology()
testFullUnitizedInternalMomentRecovery()
testFullUnitizedMemberSpanDisplacement()
testFullUnitizedResultantDisplacement()
testFullUnitizedLoadCombination()
testFullUnitizedProfileEnvelope()
```

Use project naming conventions.

=====

49. OTHER FLOW REGRESSION

=====

The primary scope is FULLY UNITIZED.

Do not unnecessarily modify:

- sliding
- casement
- spider
- unrelated flows

However, if the defect is in a common solver component and the mathematically correct fix benefits all flows, apply the shared fix carefully and run existing regression tests.

=====

50. API / JSON COMPATIBILITY

=====

Preserve the existing result contract wherever practical.

Do not change result semantics merely to reduce numerical error.

Current result areas include:

analysis
units
properties
global
mullion
stack
head
sill
transom
profiles

Preferred architecture:

Canonical solver results

↓

Facade/member classifier

↓

Envelope/governing selector

↓

Existing JSON adapter

=====

51. RESULT SEMANTICS

=====

Verify meanings of:

major

minor

MZ

MY

FY

FZ

FX

member-minimum

member-maximum

start

end

member-span

RESULTANT

Do not infer semantics solely from names.

=====

52. PROFILE ENVELOPE LOGIC

=====

For each profile category:

mullion

stack

head

sill

transom

perform:

classify members

↓

collect complete solved results

↓

evaluate required cases/combinations

↓

evaluate required stations

↓

select governing result

↓

retain member/station/load-case metadata

Do not perform a separate structural analysis per profile.

=====

53. INTERNAL STATION AND POINT-LOAD HANDLING

=====

When loads introduce discontinuities, ensure force/displacement evaluation correctly considers both sides where required.

Particular attention must be paid to:

- point loads
- partial distributed loads
- load start/end points
- element boundaries
- panel boundaries
- member boundaries

If the solver uses continuous polynomial extrema, verify that it respects discontinuity locations.

=====

54. PERFORMANCE

=====

Correctness comes before optimization.

After correctness is established, profile:

- parser time
- model construction
- stiffness assembly
- matrix factorization
- solve
- internal force recovery
- station evaluation
- envelope generation

Use sparse methods only if justified by actual model sizes/performance requirements.

=====

55. ERROR HANDLING

=====

Do not return plausible values when analysis fails.

Detect and report issues such as:

- invalid model
- unsupported command
- invalid section
- invalid property
- zero-length member
- singular structure
- mechanism
- invalid support
- invalid release
- numerical failure
- analysis cancellation
- result recovery failure

Never hide failures with zeros, NaN suppression, arbitrary fallbacks or invented values.

=====

56. SOLVER CANCELLATION

=====

Preserve/support safe cancellation behavior for long-running jobs.

Cancellation should not leave:

- locked resources
- orphan processes
- corrupted temporary state
- inconsistent job status

Do not break existing job lifecycle or abort functionality unnecessarily.

=====

57. CLEAN ARCHITECTURE

=====

Keep responsibilities separated as much as practical:

Parser

↓

Canonical model

↓

Unit normalizer

↓

Structural validation

↓

Load/BC assembly

↓

Solver

↓

Displacement recovery

↓

Internal-force recovery

↓

Envelope

↓

Facade classification

↓

Existing JSON adapter

Do not couple the solver directly to frontend UI objects.

=====

58. FULL-UNITIZED FEATURE COMPATIBILITY

=====

The final implementation **MUST** remain compatible with valid user customization including:

- increased/decreased bay count
- unequal bay lengths
- custom bay widths
- increased/decreased panel count
- unequal panel dimensions
- custom S1/S2/S3 style segment lengths
- custom stack dimensions
- custom profile assignments
- custom PRIS properties
- derived profiles
- different section stiffness
- resulting topology changes
- resulting load changes
- resulting member/node ID changes
- resulting governing result changes

This is a core requirement, not an optional enhancement.

=====

59. REQUIRED COMPARISON TABLE

=====

For every supplied sample produce a comparison table including, where present:

- Global Major BM
- Global Minor BM
- Global Major Shear
- Global Minor Shear
- Global Axial
- Global Deflection

- Mullion BM
- Mullion Deflection
- Stack BM
- Stack Deflection
- Head BM
- Head Deflection
- Sill BM
- Sill Deflection
- Transom BM
- Transom Deflection

Also compare metadata:

- member ID
- load case
- station

station unit

location

axis

member type

source

=====

60. REQUIRED ROOT-CAUSE REPORT

=====

At completion, explicitly provide:

ROOT CAUSE

What was wrong?

EVIDENCE

Which comparison/diagnostic proved it?

FIX

What code/calculation was changed?

ENGINEERING REASON

Why is the fix mathematically/structurally correct?

VALIDATION

Which samples improved?

REGRESSION

Which other flows were checked?

REMAINING DIFFERENCES

What still differs and why?

=====

61. REQUIRED CODE-CHANGE REPORT

=====

For every modified file provide:

File:

Class:

Method:

Change:

Reason:

Test coverage:

Avoid unrelated refactoring.

=====

62. REQUIRED KNOWN-LIMITATIONS REPORT

=====

List any:

- unsupported STAAD command
- unsupported load type
- unsupported release
- unsupported section property
- unsupported analysis type
- unresolved reference discrepancy
- known numerical limitation

Do not silently leave unsupported behavior.

=====

63. DEFINITION OF DONE

=====

The task is complete only when:

STRUCTURAL MODEL

- Full-Unitized .std files parse correctly.
- Variable bay counts work.
- Unequal bay dimensions work.
- Variable panel counts work.
- Variable panel dimensions work.
- Custom PRIS properties work.
- Custom profile assignments work.
- Dynamic member IDs work.
- Dynamic node IDs work.
- Dynamic topology works.

ANALYSIS

- Material data is correct.
- Section properties are correct.
- Supports are correct.
- Releases are correct.
- Loads are correct.
- Load cases are correct.
- Load combinations are correct.
- Units are correct.
- Local/global axes are correct.
- Solver formulation is correct.

RESULTS

- Major BM matches reference within justified tolerance.
- Minor BM matches reference within justified tolerance.
- Shear/axial results remain correct.
- Global deflection matches reference within justified tolerance.
- Internal member forces are correctly recovered.
- Internal governing stations are correctly identified.
- Mullion results are correct.
- Stack results are correct.
- Head results are correct.
- Sill results are correct.
- Transom results are correct.

ROBUSTNESS

- User-customized models work.
- No fixed geometry assumptions.
- No sample-specific hacks.
- No correction factors.
- No expected-value hardcoding.
- Regression tests exist.
- All supplied samples have been run.
- Other flows have been checked where shared code changed.
- Remaining discrepancies are documented.

=====

64. ABSOLUTE HARD CONSTRAINTS

=====

NEVER:

1. Hardcode a reference result.
2. Hardcode a member ID.
3. Hardcode a node ID.
4. Hardcode a bay count.
5. Hardcode a panel count.
6. Hardcode geometry.
7. Hardcode PRIS properties.
8. Hardcode profile assignments.
9. Hardcode load magnitude.
10. Hardcode load case.
11. Hardcode station.
12. Apply empirical correction factors.
13. Special-case a sample filename.
14. Modify expected JSON to make tests pass.
15. Modify reference data.
16. Return invented fallback values.
17. Hide solver failures.
18. Assume the sample topology is universal.
19. Validate only one sample.
20. Stop once one sample matches.
21. Change JSON semantics just to reduce numerical difference.
22. Hide a generator defect inside the solver.
23. Make unrelated application changes without evidence.

=====

65. MOST IMPORTANT ENGINEERING PRINCIPLE

=====

The objective is NOT:

"make one sample JSON match"

The objective is:

"correctly solve the complete supported class of Full-Unitized structural models generated by our application and reproduce the intended OpenSTAAD reference behavior within justified engineering/numerical tolerance."

The correct flow is:

User customization



Actual Full-Unitized .std



Generic STAAD parser



Canonical structural model



Correct units



Correct loads/BCs/releases



Correct FEM/frame formulation



Correct matrix solution



Correct nodal displacement



Correct member internal forces



Correct internal stations/extrema

↓

Correct load-case/combination envelope

↓

Correct global results

↓

Correct profile-specific results

↓

OpenSTAAD-compatible JSON

=====

66. FIRST-DIVERGENCE RULE

=====

For every mismatch, do not immediately change a formula.

First determine:

What is the expected input?

What is Mini-STAAD actually using?

Where do the values first diverge?

What engineering equation should govern that stage?

Is the implementation mathematically correct?

Is the reference behavior compatible with it?

Then fix the earliest incorrect stage.

=====

67. STARTING INSTRUCTION — DO NOT CODE IMMEDIATELY

=====

Before editing production code:

1. Inspect the entire repository relevant to Full-Unitized.
2. Find the Full-Unitized model generator.
3. Find Mini-STAAD parsing and model construction.
4. Find section/PRIS handling.
5. Find load generation/parsing.
6. Find supports and releases.
7. Find load cases/combinations.
8. Find stiffness assembly.
9. Find solver.
10. Find displacement recovery.
11. Find internal force recovery.
12. Find station/extrema logic.
13. Find global/profile envelope logic.
14. Inspect ALL supplied Full-Unitized .std/reference JSON pairs.
15. Run the current Mini-STAAD against ALL supplied models.
16. Produce a discrepancy matrix.
17. Identify the first numerical divergence.
18. Rank root-cause hypotheses by evidence.
19. Explain recommended fixes.
20. Only then modify production code.

Do NOT start by changing output formulas.

=====

68. REQUIRED FIRST RESPONSE FROM ASTRA

=====

Before implementation, return a detailed analysis with:

- A. Repository architecture relevant to Full-Unitized.

- B. Full-Unitized .std generation flow.
- C. Mini-STAAD parsing flow.
- D. Canonical model representation.
- E. PRIS/property handling.
- F. Load handling.
- G. Support/release handling.
- H. Load-case/combination handling.
- I. Structural element formulation.
- J. Unit handling.
- K. Displacement recovery.
- L. Internal force recovery.
- M. Station/extrema logic.
- N. Global/profile envelope logic.
- O. Actual discrepancies across all samples.
- P. First-divergence candidates.
- Q. Root-cause hypotheses with evidence.
- R. Proposed minimal code changes.
- S. Regression strategy.
- T. Risks and known limitations.

Only after this analysis should implementation begin.

=====

69. FINAL VALIDATION

=====

After implementation:

1. Run unit tests.
2. Run simple engineering benchmark tests.
3. Run every supplied Full-Unitized .std/reference JSON pair.

4. Generate actual Mini-STAAD outputs.
5. Compare against the exact reference outputs.
6. Verify numerical differences.
7. Verify governing member IDs/stations/load cases where appropriate.
8. Verify global results.
9. Verify mullion results.
10. Verify stack results.
11. Verify head results.
12. Verify sill results.
13. Verify transom results.
14. Test representative custom bay/panel/property variations.
15. Run existing other-flow regression tests when shared solver code changed.
16. Verify no expected/reference files were changed.
17. Verify no sample-specific logic was introduced.
18. Document remaining differences.

=====

70. FINAL REPORT FORMAT

=====

At the end provide:

1. ROOT CAUSE
2. EVIDENCE
3. FILES/METHODS CHANGED
4. ENGINEERING EXPLANATION
5. BEFORE vs AFTER COMPARISON
6. TESTS ADDED
7. ALL-SAMPLE VALIDATION STATUS
8. OTHER-FLOW REGRESSION STATUS
9. REMAINING DIFFERENCES

10. KNOWN LIMITATIONS

For each sample, include:

Sample

Metric

OpenSTAAD Reference

Mini-STAAD Before

Mini-STAAD After

Absolute Error

Relative Error

Reference Member

Actual Member

Reference Station

Actual Station

Reference Load Case

Actual Load Case

Status

Where a field is genuinely unavailable, report N/A instead of inventing it.

=====

71. FINAL GOAL

=====

Build a robust, generic and maintainable Full-Unitized Mini-STAAD implementation.

It must be compatible with the application's actual customizable input space:

more/fewer bays

unequal bays

- more/fewer panels
- unequal panel dimensions
- custom segment lengths
- custom stack dimensions
- custom profiles
- custom PRIS
- derived profiles
- variable topology
- variable loads
- variable member IDs
- variable node IDs

It must correctly analyze the resulting .std and generate OpenSTAAD-compatible structural results.

The solver should be generic and data-driven.

The frontend configuration must influence the .std, and the .std must drive the solver.

Do not create hidden assumptions that the frontend always generates one fixed model.

The final implementation should be:

- Generic
- Deterministic
- Numerically sound
- Unit-safe
- Testable
- Extensible
- Robust to user customization
- Compatible with existing result consumers

Free of sample-specific hardcoding

MOST IMPORTANT:

Find and fix the underlying cause of bending-moment and deflection differences.

Do not merely make the output numbers look closer.

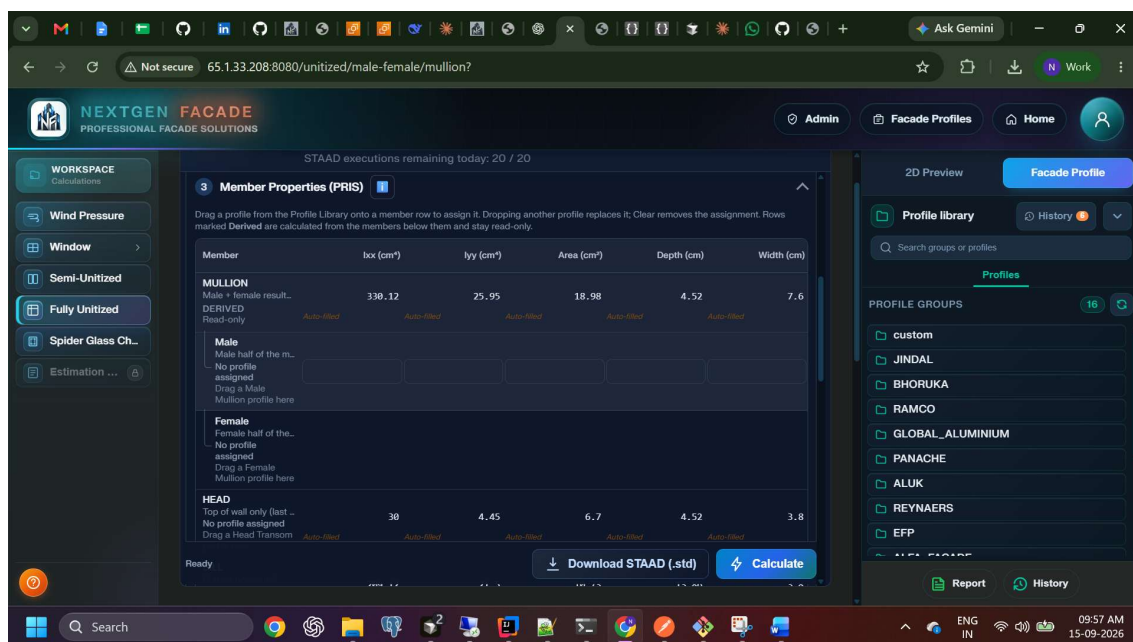
Do not apply correction factors.

Do not hardcode sample values.

Do not change reference files.

Do not stop after one model passes.

Start with repository analysis and all supplied Full-Unitized samples, then implement the minimum technically correct changes and prove the result through regression and comparison testing.



V

WORKSPACE

Calculations

Wind Pressure

Window

Semi-Unitized

Fully Unitized

Spider Glass Ch...

Estimation ...

STAAD executions remaining today: 20 / 20

assigned

Drag a Female Mullion profile here

HEAD

Top of wall only (last ...)

No profile assigned

Drag a Head Transom profile here

Auto-filled

Auto-filled

Auto-filled

Auto-filled

Auto-filled

Auto-filled

3.8

SILL

Bottom horizontal

No profile assigned

Drag a Sill Transom profile here

Auto-filled

Auto-filled

Auto-filled

Auto-filled

Auto-filled

Auto-filled

3.8

MID TRANSOM

Intermediate horizon...

No profile assigned

Drag a Mid Transom profile here

150.9

87.04

8.9

10

8

STACK TRANSOM

Inter-panel horizontal...

DERIVED

Read-only

239.12

25.95

17.43

4.52

7.60

Ready

Download STAAD (.std)

Calculate

2D Preview

Facade Profile

Profile library

History

Search groups or profiles

Profiles

PROFILE GROUPS

16

custom

JINDAL

BHORUKA

RAMCO

GLOBAL_ALUMINIUM

PANACHE

ALUK

REYNAERS

EFP

Report

History

Search

ENG IN

09:57 AM

15-09-2026

WORKSPACE

Calculations

Wind Pressure

Window

Semi-Unitized

Fully Unitized

Spider Glass Ch...

Estimation ...

STAAD executions remaining today: 20 / 20

4 Panels

+ Add panel

Panel 3

S1 1.4 m

S2 3.8 m

S3 1.000 m

Stack 0.14 m

Panel 2

S1 1.4 m

S2 3.4 m

Stack 0.14 m

Panel 1

S1 1.4 m

S2 3.4 m

Stack 0.14 m

Ready

Download STAAD (.std)

Calculate

2D Preview

Facade Profile

Profile library

History

Search groups or profiles

Profiles

PROFILE GROUPS

16

custom

JINDAL

BHORUKA

RAMCO

GLOBAL_ALUMINIUM

PANACHE

ALUK

REYNAERS

EFP

Report

History

Search

ENG IN

09:57 AM

15-09-2026

